Implementation Foundation

Process Change Impact Simulator | Week 1 GitHub Milestone

Project purpose: This project is intended to help users understand how changing one activity, approval, or condition in a business process may affect later activities. The first version uses sample workflows and graph algorithms rather than confidential company data.

1. Source Tree and Main Components

The source tree is being organized so that each part has a clear job. The Flask application handles browser requests, while the process-analysis code builds a directed graph and checks which activities are connected to a selected activity. The database layer stores sample process information. HTML templates and CSS provide the web pages, and tests check expected results.

app.py	Starts the Flask application and defines the web routes.
requirements.txt	Lists Python packages needed to run the application.
src/process_simulator/graph_builder.py	Planned module for building a NetworkX graph from activities and dependencies.
src/process_simulator/impact_analyzer.py	Planned module for finding downstream activities and checking graph structure.
src/process_simulator/simulation_engine.py	Planned module for comparing original and changed process paths.
src/process_simulator/database.py	Planned database setup and helper functions for storing process data.
templates/ and static/	HTML pages and CSS used by the browser interface.
data/	Synthetic sample workflows, such as purchase approval or employee onboarding.
tests/	Automated tests for graph analysis, simulation, database behavior, and routes.

The modules labelled planned describe the intended structure and should only be marked complete after their code has been added and tested. The exact tree in the GitHub repository should match the files that are actually present.

2. Installing Dependencies and Running the Software

The project is designed for Python 3.11 or 3.12, Flask, SQLite, and NetworkX. From the project root, use Windows PowerShell:

```
py -3.12 -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
```

```
pip install -r requirements.txt
python app.py
```

When Flask starts successfully, open http://127.0.0.1:5000 in a browser. If the current repository uses a different port or startup command, follow the command and address shown by the actual application. To run automated tests, use `pytest -q` from the project root.

The dependency file should list the packages actually used by the code. A suitable initial set for the Flask proof of concept is Flask, Flask-SQLAlchemy, NetworkX, and pytest. SQLite is included with Python, so a separate database server is not needed for local development.

3. What Currently Executes Successfully

The Week 1 report should describe only functions that have been run and checked in the repository. The planned technical foundation is a local Flask application with a sample purchase-approval workflow, NetworkX graph analysis, downstream dependency tracing, cycle detection, a simple comparison of process paths, and SQLite storage. Automated tests are intended to check key graph and simulation behavior.

Before submitting, run the application and test suite on your computer, then revise this section to match the actual results. Do not report a feature as working if its code is still planned or if the test has not been run. Include a screenshot or test output in the repository if your instructor expects evidence.

4. Changes to Earlier Design Decisions

The original proposal described a Streamlit interface. The implementation plan has changed to Flask with HTML and CSS because a traditional web application gives clearer separation between the interface, routes, process logic, and database. SQLite is planned for local data storage, and NetworkX remains the main library for representing dependencies and finding downstream effects. The project continues to use Python and synthetic workflows. No external business system or confidential dataset is required.

5. Planned Week 2 Functionality

- Finish organizing the source code under the agreed project structure.
- Build and validate a directed graph from saved activities and dependencies.
- Add a clear way to select an activity and display its downstream activities.
- Improve the sample purchase-approval workflow and keep its data separate from proposed changes.
- Implement or extend simulation comparison for an approval-threshold change.
- Add tests for sequential paths, branching paths, missing activities, and circular dependencies.
- Check database initialization and make sure existing sample data is not overwritten during normal startup.
- Record test results and capture screenshots for the Week 2 milestone.